

CS 593 RL1 Fall 2025

Assignment 3: Policy Gradients & Actor Critic

Purdue University

Due November 11 2025, 11:59 PM

Installation

Setting up a Conda Environment

1. Open a terminal/command prompt and create a new environment:

```
conda create -n cs593rl python=3.9 -y
```

2. Activate the environment:

```
conda activate cs593rl
```

3. Navigate to the directory where you unzipped the code and install the necessary Python packages from `requirements.txt`:

```
pip install -r requirements.txt
```

This assignment uses two Gymnasium environments: LunarLander-v2 and CarRacing-v2. Refer to the documentation pages linked above for information on the state space, action space, and reward function (though in this assignment rewards are only reported as part of the evaluation).

Part 1: Implement Agents [6 pts]

We have provided a starter implementation for three agents REINFORCE Soft Actor-Critic (SAC), and Proximal Policy Optimization (PPO), with some functionality unimplemented. You have the option of running the code either on your own machine, on a cloud based service (such as Google Colab), or on RCAC Scholar.

Use the concepts introduced in class along with the Gymnasium/PyTorch documentation to fill in the blanks in the code marked with `TODO`.

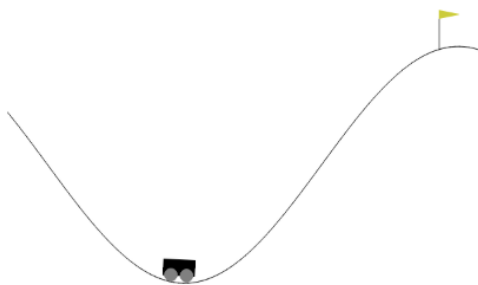


Figure 1: MountainCarContinuous-v0

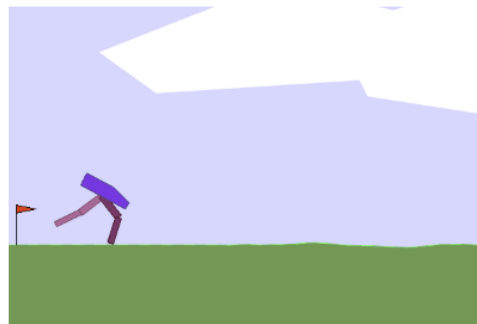


Figure 2: BipedalWalker-v3

We will use two environments for this homework: BipedalWalker-v3, MountainCarContinuous-v0. For information related to the observation space, reward metrics etc., visit the environment website:

https://gymnasium.farama.org/environments/box2d/bipedal_walker/

https://gymnasium.farama.org/environments/classic_control/mountain_car_continuous/

A. REINFORCE [2 pts]

Implement REINFORCE agent following the TODO sections in the `reinforce.py` file.

For REINFORCE agent the following TODO sections will be completed:

1. `select_action` function in `REINFORCEAgent`
2. `calculate_discounted_returns` function in `REINFORCEAgent`
3. `train_reinforce` function in `REINFORCEAgent`

The details and hints about what to do will be in the code.

Once the implementation is complete, here is how you can run each task: Check `main.py` file for all the args that can be used.

```
python main.py --reinforce --env MountainCarContinuous-v0
```

```
python main.py --reinforce --env BipedalWalker-v3
```

Add the data that you collected (loss, reward) for each environment to your PDF report using Tensorboard by running `tensorboard --logdir runs` and navigating to `http://localhost:6006/` in your browser.

In addition, analyze the training performance and report. If you are unable to find an optimum policy, give an explanation on the results of your training in your own words. It would be expected if your policy does not converge. In this section, you won't be evaluated based

on the quality of your policy, but on your analysis of your results.

Bonus Challenge: After you are done with the homework, you can tweak the REINFORCE algorithm's hyperparameter and add different components to make it work well under the environments. If you can find a set of hyperparameters/implementations that the policy converges in both environments, you will get up to +5 points to replace missed points in other parts (this will not give more than 10/10). If you complete this challenge, add your experiments, set of hyperparameters, and your changes in the code to the pdf, and your data folders of those experiments in your submission. You are still free to share your findings even if it doesn't work under the Bonus header, explaining what you tried and what didn't work.

B. Soft Actor-Critic [2 pts]

Implement SAC agent following the `TODO` sections in the `sac.py` file.

For SAC Agent the following will be completed:

1. `sample` function in `PolicyNetwork`
2. `update_parameters` function in `SACAgent`

The details and hints about what to do will be in the code.

Add the data that you collected (loss, reward) for each environment to your PDF report using Tensorboard.

C. PPO [2 pts]

Implement PPO agent following the `TODO` sections in the `ppo.py` file.

For PPO Agent the following will be completed:

1. `compute_gae` function in `PPOAgent`
2. `update_policy` function in `PPOAgent`

The details and hints about what to do will be in the code.

Add the data that you collected (loss, reward) for each environment to your PDF report using Tensorboard.

Collective Analysis for Part 1: Analyze all the results that you collected so far and compare them. Discuss your findings, your hypothesis, and your analysis so far. Choose what plots you want to show and support your analysis. You are free to express uncertainty in your analysis, for example, I have found that X result changed between those Y and Z agents, this could potentially be related to the exploration bonus. You will not lose points if your reasoning was off, but try to express your reasoning by giving as much evidence as possible.

Part 2. Agent and Hyperparameter analysis [4 pts]

Experiment with the given hyperparameters and observe how the values affect the performance of the agents. You will have to produce meaningful findings and report your findings.

A. Hyperparameter tuning and creating a new list of hyperparameters

You will choose one agent of your choice (e.g. PPO or SAC) with only one environment and tune extensively. Out of the available hyperparameters for the choice of your agent, do at least 3 sets of analyses for that agent, although more analyses are welcomed.

You will choose an agent and a list of hyperparameters that you think will influence the learning significantly.

On PPO, some of the hyperparameters you can test are: the clip_ratio, entropy_coef, gae_lambda, and batch_size, and learning_rate.

And for SAC, some of the hyperparameters you can test are: learning_rate, tau, replay_buffer_size, batch_size, and gamma.

Feel free to test other hyperparameters; you are free to create your list of hyperparameters into your analysis and test them. For every hyperparameter you should do 3-4 values to find an optimum set (e.g clip_ratio = 0.1, 0.2, 0.25, 0.3). Run all the hyperparameter analysis with only one environment.

Here are some examples on how to run different agents and hyperparameters. You can change these values by using the following commands:

```
python main.py --ppo --env BipedalWalker-v3 --lr=1e-5
python main.py --sac --env MountainCarContinuous-v0 --gamma=0.9
```

After running hyperparameter tests, you will see that one of those values will give better result (e.g. out of 4 values on clip_ratio I found 0.2 to be the best). You will write those down to create an updated list of hyperparameters.

Share your results for the hyperparameters and your final hyperparameter list in the report. Discuss on your findings, which tests were important, which hyperparameters influenced the agent how, etc. Add all your thought processes and analysis.

Tip: You can toggle certain runs as visible or invisible by clicking the checkbox next to run name on the left of the Tensorboard window.

B. Running new hyperparameter list and comparing with the old results

With your new hyperparameter list, run your agent on the environment that you chose and compare the new results with the base results from Part 1. For example, if you chose PPO and Bipedal for your analysis, run the updated hyperparameter list for Bipedal, and then

compare the changes with your old experiment for Bipedal from part 1.

Here you should discuss (1) how your findings from hyperparameter tuning reflected in your experiment, (2) is there is a significant improvement, if so, which hyperparameters do you think influenced that, if not why do you think there wasn't an improvement, (3) which hyperparameters worked well, and which update was crucial. (4) and any more of your thought process and your hypotheses.

You can also go back to part a, and do more analysis to find a more optimal list of hyperparameters if you think you can improve further. High-quality analysis, findings, and discussion are what we look for full grades in this section.

Submission Instructions

You may download/screenshot the Tensorboard figures and include them directly in the PDF report and write your analysis under them. Include your name to the PDF report. Decouple your work via headers for the report (Part 1A, Part 2B etc.)

In order to turn in your code and experiment logs, create a zip folder that contains the following:

- Your PDF report. (Include your name into the report and the report file)
- Your completed code files.
- Your runs from Part 1, and your Part 2b run in "selected_runs" (to keep the submission file sizes small.)

Upload the resulting zip file with submission file named as

`firstname_lastname_hw3.zip`

to Brightspace.